Appendix of "QTem: Bounded-Error Compressed Data Movement for Temporal Graph Neural Network Training"

Weiwei Ye
The University of Tokyo
Japan
wwye@g.ecc.u-tokyo.ac.jp

Yankai Chen
MBZUAI
United Arab Emirates
yankaichan3@gmail.com

Zhen Wang
Huzhou Normal University
China
zhenwangrs@gmail.com

Dongyuan Li
The University of Tokyo
Japan
lidy@csis.u-tokyo.ac.jp

This appendix accompanies the paper. Sections, tables, figures, equations, and statements that are not part of this document refer to the paper; tables and figures continue its numbering.

A PROOFS

We use the notation of Section 4, where the proofs of Theorem 4.1, Corollary 4.2, and Proposition 4.4 are given.

Proof of Proposition 4.3. Let $\hat{\boldsymbol{\alpha}}$ denote the attention weights computed from the decoded inputs, that is, the softmax of $\hat{\mathbf{z}}$. Then

$$\mathbf{h} - \hat{\mathbf{h}} = \sum_n (\alpha_n - \hat{\alpha}_n)\, \mathbf{W}_V \mathbf{x}_n + \sum_n \hat{\alpha}_n\, \mathbf{W}_V (\mathbf{x}_n - \hat{\mathbf{x}}_n).$$

The second sum is a convex combination, so its norm is at most $\|\mathbf{W}_V\|_2\, \varepsilon$. For the first,

$$\Big\| \sum_n (\alpha_n - \hat{\alpha}_n) \mathbf{W}_V \mathbf{x}_n \Big\|_2 \le \|\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}\|_1 \max_n \|\mathbf{W}_V \mathbf{x}_n\|_2.$$

The Jacobian of the softmax at any point is $\mathrm{diag}(\boldsymbol{\sigma}) - \boldsymbol{\sigma}\boldsymbol{\sigma}^\top$, and for every $\mathbf{v}$ it satisfies $\sum_i \sigma_i |v_i - \boldsymbol{\sigma}^\top \mathbf{v}| \le 2\|\mathbf{v}\|_\infty$; integrating along the segment from $\mathbf{z}$ to $\hat{\mathbf{z}}$ yields $\|\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}\|_1 \le 2\|\mathbf{z} - \hat{\mathbf{z}}\|_\infty$. Finally, $|z_n - \hat{z}_n| \le \|\mathbf{q}\|_2 \|\mathbf{W}_K\|_2\, \varepsilon / \sqrt{d'}$ by the Cauchy–Schwarz inequality. Combining the three bounds proves the claim. If the query is itself computed from a decoded root feature, the same argument adds a term proportional to its input error, again independent of k. □

B ADDITIONAL ACCURACY RESULTS

Table 4 gives the standard deviations over seeds of the AP in Table 2 and, in its last column, the AP of the full-precision pipeline trained with the same settings and seeds as QTem. Table 5 lists the paired runs on the benchmark graphs and Table 6 those on the two real-feature graphs behind Figure 4. Both arms of a pair share the code base, the initialization, the sampler, and the negatives; only the representation of the static features differs. 8-bit codes do not make training less stable: the median seed-to-seed standard deviation of QTem over the cells is 0.20 points, against 0.24 at full precision. Wikipedia and Reddit use TGL's settings for these graphs [2] (batch size 600, learning rate 10^{-4}, ten epochs, dropout 0.2 for TGN and 0.1 otherwise) with the same single-layer models and checkpoint rule. The control arm trains the same models with every static feature replaced by zero; it confirms that the models exploit the information in the LIWC features, most clearly TGN and TGAT on Reddit, and QTem preserves this information.

Stack-Overflow-APAN. This configuration is the least stable one: at full precision, its test AP ranges from 67.9 to 78.7 across seeds, and it yields the largest paired difference (mean −7.16 points, $p = 0.25$). Two controls show that this difference stems from where training ends, not from the 8-bit inputs. A repeated full-precision run of seed 0 with identical code, configuration, seed, and inputs ends 3.9 points from the original, because nondeterministic memory updates make two runs diverge from the first epoch on. Evaluating the trained models of both pipelines for seeds 0 and 1 with the inputs of the other pipeline moves their test AP by at most 0.44 points.

Ranking with 49 negatives. We re-evaluated all 90 paired checkpoints of Tables 5 and 6 on the test split with 49 random negatives per positive, each with the representation it was trained with, and computed the mean reciprocal rank of every positive among its 50 candidates (Table 7). After Holm's correction for the 15 cells, no cell differs significantly (smallest adjusted $p = 0.12$), and over all 45 pairs, the MRR of QTem differs from full precision by +0.4 points on average (95% CI [−1.5, 2.3]).

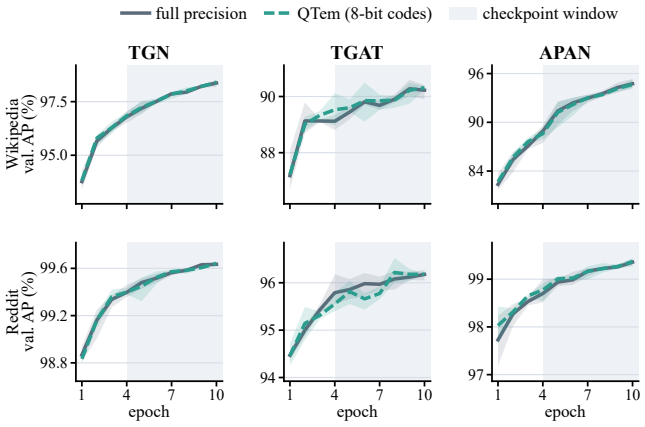


Figure 9: Training dynamics on real features: validation AP after every epoch for the full-precision pipeline and QTem (mean over three seeds; bands: min–max). The shaded region is the window from which the checkpoint is selected.

Training dynamics. Figure 9 follows the validation AP of the real-feature runs through all ten epochs. In every panel, the largest gap between the mean curves of the two pipelines, at most 0.54 points, is smaller than the spread across seeds of the full-precision pipeline at the same epoch.

C LOCK-STEP REPLAY DETAILS

Table 8 summarizes the LastFM replay of Section 5.4 per model and codec, averaged over the three trained models. The input error is the relative error of the event features that enter messages and mailboxes; the logit deviation and the share of flipped link predictions

Table 4: Test AP (%) in Table 2: mean ± standard deviation over the seeds 0, 1, and 2. †Paired-run settings (GDELT: seeds 0, 1, and 7): QTem and the full-precision pipeline (fp32) of the same runs.

Graph	Model	TGL	SIMPLE	ETC	QTem†	fp32†
LastFM	TGN	80.44±1.88	81.68±0.26	80.61±0.16	84.07±0.57	83.86±0.98
	TGAT	67.56±0.63	67.60±0.67	67.37±0.56	69.61±0.48	69.52±0.50
	APAN	69.95±0.47	68.72±3.31	69.46±0.15	69.20±0.19	69.45±0.24
Wiki-Talk	TGN	92.97±0.12	84.14±0.10	93.16±0.39	89.94±0.89	90.03±3.28
	TGAT	86.16±0.09	82.15±0.13	86.06±0.06	86.87±0.07	86.85±0.08
	APAN	82.73±3.22	76.19±0.53	84.99±2.63	86.43±0.63	86.62±0.38
Stack-Overflow	TGN	84.58±1.69	86.93±0.50	86.45±0.92	85.83±0.91	85.93±0.04
	TGAT	87.71±0.06	87.70±0.27	87.74±0.17	87.66±0.05	87.44±0.08
	APAN	67.96±2.01	72.80±0.07	69.15±0.90	64.78±2.05	71.94±5.85
GDELT	TGN	97.41±1.04	96.49±2.82	97.87±0.24	97.23±0.89	97.84±0.46
	TGAT	98.07±0.01	98.01±0.01	98.06±0.00	98.08±0.01	98.08±0.01
	APAN	97.08±0.37	96.79±0.42	96.99±0.08	97.52±0.35	97.48±0.33

Table 5: Per-seed test AP (%) on the benchmark graphs. LastFM, Wiki-Talk, and Stack-Overflow: paired runs of the full-precision pipeline and QTem from the same initialization, sampler, and negatives; Δ: mean paired difference with the p-value of a paired t-test. GDELT: bit-plane packing is lossless, so QTem reads exactly the full-precision inputs (*: seed 7).

		Seed 0		Seed 1		Seed 2*		Mean			
Graph	Model	fp32	QTem	fp32	QTem	fp32	QTem	fp32	QTem	Δ	p
LastFM	TGN	82.78	83.79	84.10	83.69	84.69	84.72	83.86	84.07	+0.21	0.66
	TGAT	69.72	69.62	69.89	70.09	68.94	69.13	69.52	69.61	+0.10	0.44
	APAN	69.44	69.02	69.21	69.40	69.69	69.18	69.45	69.20	−0.25	0.38
Wiki-Talk	TGN	86.25	90.71	91.72	90.15	92.13	88.96	90.03	89.94	−0.10	0.97
	TGAT	86.94	86.87	86.81	86.80	86.80	86.93	86.85	86.87	+0.02	0.81
	APAN	86.21	87.06	86.96	85.80	86.70	86.45	86.62	86.43	−0.19	0.77
Stack-Overflow	TGN	85.93	86.85	85.89	85.53	85.97	85.10	85.93	85.83	−0.10	0.87
	TGAT	87.44	87.66	87.35	87.62	87.52	87.72	87.44	87.66	+0.23	0.01
	APAN	69.22	64.76	78.65	62.73	67.95	66.83	71.94	64.78	−7.16	0.25
GDELT	TGN	98.17		97.11		96.40		97.23		lossless	
	TGAT	98.09		98.07		98.08		98.08		lossless	
	APAN	97.15		97.56		97.84		97.52		lossless	

Table 6: Paired runs on real features (test AP, %; mean ± standard deviation over three seeds). Δ: mean paired difference to full precision, with the p-value of a paired t-test; 4-bit: four-bit codes with one scale per row.

			QTem, 8-bit		4-bit	no features	
Graph	Model	fp32	Δ	p	Δ	Δ	p
Wikipedia	TGN	98.42±0.19	+0.08	0.51	−0.03	+0.03	0.75
	TGAT	89.87±0.01	+0.04	0.71	+0.11	−0.15	0.32
	APAN	94.45±0.36	+0.03	0.93	+0.23	−0.66	0.04
Reddit	TGN	99.57±0.05	+0.05	0.37	+0.01	−0.97	0.007
	TGAT	96.52±0.02	+0.07	0.61	+0.09	−4.53	0.003
	APAN	96.88±1.45	+2.07	0.14	+0.50	−1.36	0.57

compare every batch with the deterministic full-precision copy; and the memory deviation is the relative Frobenius distance of the whole memory table, sampled every two batches. The last row of each block compares two copies of the unmodified full-precision pipeline. The ratio of memory deviation to input error is 0.14 for TGN and 0.068 for APAN, nearly identical for all three codecs: the memory error is a linear response to the input error with a model-specific gain below one, which the mean-square gains of the trained updaters predict (Appendix D).

Real features. Figure 10 and Table 9 list the replay of Wikipedia and Reddit behind Section 5.4. Each row averages the three models trained at full precision with different seeds; deviations are measured against the deterministic full-precision copy over the test segment, and ΔAP is the change of the test AP of the replayed model. With 8-bit codes, no replayed model changes its test AP by more than 0.006 points, and fewer than 0.12% of link predictions change sign, whereas destroying the features changes predictions substantially, above all on Reddit-TGAT, whose test AP drops by 23 points with zeros and by 12 with shuffled rows. Four-bit codes move the logits of the Reddit models by up to 89%, which is why QTem uses 8-bit codes.

D CONTRACTION OF TRAINED MEMORY UPDATERS

Section 4 examines the premise of Proposition 4.4 on trained models. We replay the full LastFM stream through the TGN and APAN

Table 7: Mean reciprocal rank (%) with 49 random negatives per positive on the test split (mean ± standard deviation over three seeds), and the mean paired difference with its 95% confidence interval and paired t-test.

Graph	Model	fp32	QTem	Δ [95% CI]	p
LastFM	TGN	37.83±1.28	38.32±0.59	+0.49 [−2.46, +3.43]	0.55
	TGAT	20.29±0.28	20.42±0.15	+0.13 [−0.28, +0.54]	0.30
	APAN	17.34±0.20	17.01±0.30	−0.33 [−0.99, +0.33]	0.16
Wiki-Talk	TGN	55.90±11.12	57.91±3.32	+2.01 [−33.8, +37.9]	0.83
	TGAT	52.27±0.09	52.35±0.31	+0.08 [−0.60, +0.77]	0.66
	APAN	46.39±0.76	46.16±1.86	−0.22 [−3.25, +2.80]	0.78
Stack-Overflow	TGN	50.03±0.13	48.79±1.84	−1.24 [−5.71, +3.23]	0.35
	TGAT	54.35±0.17	55.05±0.26	+0.70 [+0.43, +0.97]	0.01
	APAN	23.47±6.55	16.68±0.26	−6.79 [−23.4, +9.86]	0.22
Wikipedia	TGN	79.63±2.44	79.50±1.50	−0.13 [−4.55, +4.29]	0.91
	TGAT	59.38±6.87	55.72±6.04	−3.66 [−35.7, +28.4]	0.67
	APAN	58.08±1.56	58.47±4.44	+0.39 [−6.76, +7.54]	0.84
Reddit	TGN	91.27±0.72	91.85±0.78	+0.58 [−2.02, +3.18]	0.44
	TGAT	74.25±0.26	74.33±0.77	+0.08 [−2.46, +2.62]	0.91
	APAN	68.84±10.61	82.40±2.47	+13.6 [−9.95, +37.1]	0.13
All 45 pairs				+0.38 [−1.55, +2.30]	0.70

Table 8: Lock-step replay on LastFM (mean over three trained models; deviations relative, in %).

		Input	Logit	Flips	Memory		|ΔAP|
Model	Variant	err.	dev.		median	max	(pts)
TGN	QTem, $b = d$	0.65	0.21	0.10	0.09	0.57	≤0.002
	QTem, $b = 4$	0.31	0.10	0.05	0.04	0.27	≤0.001
	4-bit, $b = d$	11.7	3.79	1.80	1.69	10.2	≤0.064
	FP32 vs. FP32	0	3.49	1.36	6.03	59.0	≤0.040
APAN	QTem, $b = d$	0.65	0.02	0.00	0.04	1.02	≤0.003
	QTem, $b = 4$	0.31	0.02	0.00	0.02	4.72	≤0.001
	4-bit, $b = d$	11.7	0.41	0.01	0.81	8.68	≤0.018
	FP32 vs. FP32	0	0.42	0.01	1.14	18.4	≤0.009

models trained with seed 0 (Table 5), with duplicate memory writes resolved deterministically as in Section 5.4. On every 25th batch, for up to 500 (TGN) or 100 (APAN) positive root nodes, we compute by automatic differentiation the Jacobian of the memory update with respect to every memory block that it reads and to the event features. TGN's GRU reads the node's own memory, as hidden state and as the first part of its message (**A** sums both Jacobians), the other endpoint's memory ($\mathbf{B}_1$), and the event feature (**C**). APAN's transformer updater reads the node's memory (**A**) and the two endpoint memories and the event feature of each of its ten mailbox entries (20 blocks $\mathbf{B}_j$, ten blocks of **C**). Our re-implementations reproduce the modules' outputs up to 9.5×10^{-7}. Table 10 summarizes the constants over all sampled updates of nodes whose memory is no longer empty.

Both updaters contract in mean square: g is at most 0.84 for TGN, and for APAN it exceeds one in a single sampled update. The gains do not drift over the stream; their medians are 0.80, 0.80, and 0.80 for TGN and 0.29, 0.28, and 0.28 for APAN in the training, validation, and test segments. With the root-mean-square values of the event features (1.00 for both models) and of the memories (0.74 for TGN, 0.78 for APAN), the steady state $l\,\sigma_x/\sqrt{1-g^2}$ predicts the ratios in

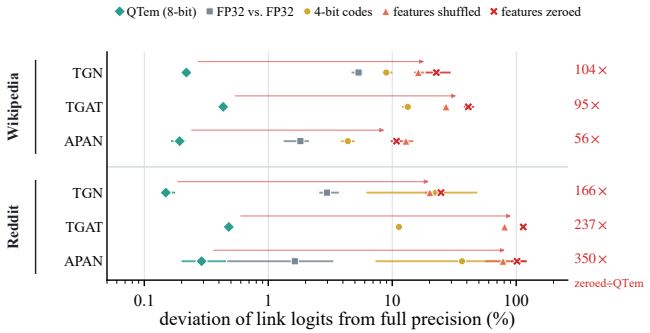


Figure 10: Feature sensitivity of trained models on real features: relative deviation of the link logits from the full-precision copy over the test segment of a lock-step replay (mean over three trained models; bars: min–max) when the models read QTem codes, 4-bit codes, shuffled feature rows, or zeros instead of the original features, and between two runs of the unmodified memory-based models. Right: ratio between zeroed features and QTem.

the last rows from the median constants, in agreement with the replay within a factor of 1.3 for TGN and within 2% for APAN.

E COMPARISON WITH SIMPLE

Setup. SIMPLE [1] decides offline, from the access pattern of one sampled epoch, which rows of each table stay in a GPU buffer during which mini-batches, and updates the buffer between batches; all other rows are gathered from host memory at full precision. For our data-path comparisons, we ran it with the same model configurations as the paired runs and the feature tables of our other runs. The buffer holds 10% of the data, SIMPLE's default, except on GDELT, where it receives the 4.4 GB that QTem-G occupies. We completed the APAN path of its training and evaluation loops as in TGL and corrected three errors of its released buffer management that corrupt buffered rows: buffer slots are assigned by positions in a masked array but checked against the length of the unmasked one, evicted node memories and mailboxes are not written back, and batches smaller than the buffer are assembled inside it. With the corrections, which do not change the placement plan, every buffered feature row matches its host copy, and SIMPLE trains TGAT exactly like TGL: every corrected SIMPLE run on TGAT that shared its seed with a TGL run reached the same test AP. Table 2 lists SIMPLE as released; the corrections only add work, so its times there favor SIMPLE.

Results. We break SIMPLE's epoch times in our data-path comparisons down using its own timers. On Stack-Overflow-TGAT, SIMPLE takes 634 s per epoch against 409 s for FP32 and 269 s for QTem-G in the same round. Its buffer reduces the rows gathered from host memory to 64 s of gathering per epoch, but managing the buffer, that is, deriving the buffer contents of each batch from the plan, maintaining flags and indicators, and updating the buffer, costs 183 s, more than the entire feature path of FP32. On GDELT-TGAT, with a buffer as large as the QTem records, SIMPLE takes 712 s against 621 s for FP32: the buffer leaves 159 s of gathering per

Table 9: Lock-step replay on real features (mean over three trained models). Dev.: relative deviation of the link logits from the full-precision copy over the test segment (%); flips: share of link predictions whose sign changes (%); ΔAP: change of the test AP (points; for QTem the largest magnitude over the three models). FP32 vs. FP32 compares two copies of the unmodified models; for TGAT, which has no memory, they agree exactly.

		QTem, 8-bit			4-bit codes		shuffled rows		zeros		FP32 vs. FP32
Graph	Model	dev.	flips	|ΔAP|	dev.	ΔAP	dev.	ΔAP	dev.	ΔAP	dev.
Wikipedia	TGN	0.22	0.024	0.001	8.9	−0.06	16.2	−0.17	22.7	−0.43	5.35
	TGAT	0.43	0.052	0.005	13.3	−0.26	27.2	−0.83	41.1	−2.88	0
	APAN	0.19	0.011	0.006	4.4	−0.32	12.9	−0.54	10.8	−0.84	1.82
Reddit	TGN	0.15	0.009	<0.001	22.4	−1.75	20.1	−0.35	24.7	−0.37	2.97
	TGAT	0.48	0.112	<0.001	11.3	−0.25	80.7	−12.04	114.0	−23.13	0
	APAN	0.29	0.037	0.004	36.6	+0.90	78.4	−1.48	101.5	−2.49	1.64

Table 10: Jacobian constants of trained memory updaters on LastFM: worst-case gain $\rho = \|\mathbf{A}\|_2 + \sum_j \|\mathbf{B}_j\|_2$ (for TGN the smaller of this and $\sqrt{2}\,\|[\mathbf{A}\,\mathbf{B}_1]\|_2$), own-memory block $\|\mathbf{A}\|_2$, feature gain $L = \sum \|\mathbf{C}\|_2$, mean-square gains g and l (Section 4), and the ratio of relative memory deviation to relative input error predicted by the mean-square steady state and measured in the lock-step replay (Table 8). Median (99th percentile, maximum).

	TGN	APAN
Sampled updates	25,427	5,086
ρ	1.28 (1.86, 2.24)	3.07 (5.49, 20.3)
$\|\mathbf{A}\|_2$	1.03 (1.23, 1.40)	0.83 (1.72, 10.5)
L	0.24 (0.53, 0.66)	0.85 (1.31, 3.53)
g	0.80 (0.82, 0.84)	0.29 (0.45, 1.28)
Share with $g < 1$	100%	99.98%
l	0.049 (0.099, 0.23)	0.052 (0.086, 0.21)
Ratio, predicted	0.11	0.069
Ratio, measured	0.14	0.068

epoch, and managing it adds another 112 s. On Stack-Overflow-TGN, where the buffer also holds node memories and mailboxes, SIMPLE takes 1115 s against 615 s for FP32, and buffer management costs 386 s per epoch. In each of the three large configurations, the single SIMPLE epoch took longer than every one of the four FP32 epochs.

F MICROBENCHMARK DETAILS

The microbenchmark of Figure 11, summarized in Section 5.6, fetches R distinct, sorted row IDs drawn uniformly from tables of 4–24 million rows, i.e., far larger than the CPU caches, and synchronizes the GPU after every stage. Each variant is repeated 25 times for $R < 2^{18}$ and 10 times otherwise after three warm-up runs, and we report medians. Table 11 lists the latency at $R = 2^{18}$, including two variants that the figure omits. With pinned staging buffers, full-precision fetches become 2.7–3.5× faster, and QTem keeps its lead at equal copy mode: 2.2–3.7× for dense tables and 4.7–6.3× for binary tables. Storing codes and scales in two separate arrays and gathering them with the pipeline's advanced indexing, as a direct port of a row-wise 8-bit quantizer would do, is 1.1–1.5× slower than the fixed-width records, almost entirely because of the host-side gather. GPU-resident records remove the host gather altogether: at $R = 2^{14}$, close to the per-batch row counts of Stack-Overflow, a fetch from the dense $d = 172$ table with four-entry blocks takes 5.3 ms at full precision, 0.7 ms with a pinned staging buffer, and 0.15 ms from GPU-resident records.

Table 11: Fetch latency (ms) at $R = 2^{18}$, median of repeated runs. "pin": pinned staging buffer; "sep.": codes and scales in separate arrays gathered with advanced indexing (binary tables: packed rows gathered with advanced indexing).

	full precision			QTem			
Table	FP32	+RG	+pin	sep.	QTem-H	+pin	QTem-G
dense, $d = 128$	58.7	63.8	20.8	27.9	19.1	9.4	0.91
dense, $d = 172$	81.8	75.5	29.9	32.0	22.5	8.1	1.01
dense, $d = 172$, $b = 4$	81.8	75.5	29.9	42.0	34.2	11.9	1.65
dense, $d = 768$	335.4	354.4	101.7	111.9	96.3	37.1	3.83
binary, $d = 182$	96.4	95.0	32.2	11.9	9.5	6.8	1.60
binary, $d = 413$	209.9	209.2	60.7	14.3	13.0	9.6	2.28

G SCALE RANGE AND BINARY32 SCALES

Theorem 4.1 requires normal binary16 scales, which every block of magnitude between $Q \cdot 2^{-14}$ and $Q \cdot 65504$ receives. We tested the codec in isolation on adversarial inputs: all-zero blocks, blocks with magnitudes of 10^{-40}, 10^{-6}, 1, 10^{7}, and close to the binary32 maximum, and non-uniform blocks. All-zero blocks decode exactly, and blocks too small for a normal binary16 scale decode with an absolute error of at most their magnitude. Binary32 scales extend the bound to every block whose scale is a normal binary32 number, that is, of magnitude at least $Q \cdot 2^{-126} \approx 1.5 \times 10^{-36}$ for 8-bit codes: the product of a code and a scale then adds one rounding, and the argument of Theorem 4.1 yields $|x_{ik} - \hat{x}_{ik}| \le \hat{s}_{ij}(\frac{1}{2} + (2Q + \frac{1}{2})u)$. The largest error we measured over all regimes with normal binary32 scales is $0.49999\,\hat{s}_{ij}$. Binary32 scales cost two more bytes per block: with one scale per row, the reduction for $d = 128$, 172, and 768 changes from 3.94, 3.95, and 3.99× to 3.88, 3.91, and 3.98×. Binary16 scales suffice for the tables we use: no scale of the two LIWC tables and of one million rows of benchmark-style features leaves the binary16 normal range, and only 4 of 192 million four-entry blocks of the MAG sample underflow.

REFERENCES

[1] Shihong Gao, Yiming Li, Xin Zhang, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. SIMPLE: Efficient Temporal Graph Neural Network Training at Scale with

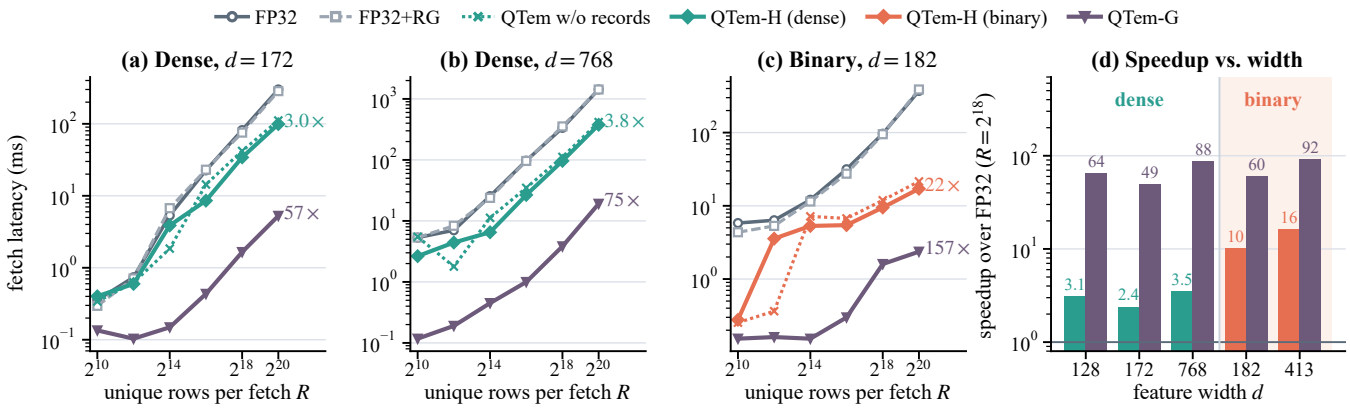


Figure 11: Latency of fetching R deduplicated rows into GPU memory as decoded rows (median of repeated runs). (a)–(c) Latency against R; labels give the speedup over FP32 at $R = 2^{20}$. (d) Speedup over FP32 at $R = 2^{18}$ per table width.

Dynamic Data Placement. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–25. https://doi.org/10.1145/3654977

[2] Hongkuan Zhou, Da Zheng, Israt Nisa, Vasileios Ioannidis, Xiang Song, and George Karypis. 2022. TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1572–1580. https://doi.org/10.14778/3529337.3529342